Guia tecnico das tecnologias do projeto

Como cada tecnologia aparece na TSZR15, como elas se conectam e quais estruturas de codigo e banco sustentam a loja.

Campo	Valor
Projeto	tszr15-whatsapp-store
Versao do app	0.1.0
Branch	perf/images-and-cache
Commit base	ad721048
Gerado em	09/07/2026 03:04
Fonte	Arquivos locais do repositorio, package.json e supabase/migrations

Escopo

Este PDF documenta as tecnologias que aparecem no codigo atual. Integracoes listadas em docs/APIS-NECESSARIAS.md entram em uma secao separada como tecnologias planejadas, nao como recursos ja implementados.

1. Visao geral

A TSZR15 e uma loja de pecas e acessorios para Yamaha R15. O frontend e feito em Next.js e React; o backend fica distribuido entre rotas API, Server Actions, proxy de sessao e funcoes RPC no Supabase/Postgres. O fechamento comercial atual usa WhatsApp Business por link wa.me com mensagem montada pelo backend.

Estrutura principal do repositorio

```
app/
  page.js, catalogo/page.js, produto/[slug]/page.js
  api/catalog/route.js, api/checkout/whatsapp/route.js
  auth/actions.js, admin/actions.js, conta/actions.js
src/
  catalog/       dados, validacao e leitura do catalogo
  components/    componentes React da vitrine, carrinho, formularios e admin
  checkout/      totais, cupom, pedido, mensagem WhatsApp
  lib/supabase/  clientes browser/server/service_role
  admin/         sessao admin, pedidos, catalogo, analytics
  tracking/      visao publica de rastreio
supabase/migrations/
  tabelas, RLS, storage buckets e RPCs do banco
tests/
  node:test para catalogo, checkout, rate limit, auth e utilitarios
scripts/
  validacao e sincronizacao do catalogo
```

O codigo segue uma separacao por responsabilidade: app/ roteia a experiencia; src/ contem dominio e componentes; supabase/ define persistencia.

Fluxo resumido de compra

```
Cliente abre catalogo
  -> Next.js Server Component busca catalogo publicado no Supabase
  -> React Client Component filtra, escolhe produto e salva carrinho no localStorage
  -> Carrinho envia POST /api/checkout/whatsapp
  -> rota API valida itens contra o catalogo confiavel do servidor
  -> backend calcula total/cupom/frete e chama RPC create_checkout_order
  -> Supabase grava orders, order_items, payments e audit_logs
  -> API retorna whatsappUrl
  -> navegador abre https://wa.me/<numero>?text=<mensagem>
```

Inventario das tecnologias em uso

Tecnologia	Onde fica	Funcao no projeto
Next.js App Router	app/, app/api, middleware.js	Rotas, Server Components, API routes, revalidacao e proxy de sessao.
React 19	src/components	Estado local, eventos, carrosseis, busca, carrinho e formularios.
Supabase SSR/JS	src/lib/supabase	Auth, queries, RPC, storage e service_role no servidor.
Postgres + RLS + PL/pgSQL	supabase/migrations	Modelo relacional, politicas de acesso, triggers e funcoes transacionais.
WhatsApp wa.me	src/checkout/whatsapp.js	Gera URL de atendimento com mensagem codificada.
ViaCEP	src/customer/cep-lookup.js	Preenche endereco a partir do CEP.
Pino	src/lib/logger.js	Logs estruturados com redacao de dados sensiveis.
Vercel Speed Insights	app/layout.js	Metrica de performance no layout raiz.
Node test runner	tests/*.mjs	Testes sem framework extra usando node:test.
ESLint + Prettier	eslint.config.mjs, package.json	Padrao de qualidade e formatacao.
GitHub Actions + Gitleaks	.github/workflows	Varredura de secrets em push/PR.

Dependencias declaradas

Pacote	Versao em package.json	Tipo
@supabase/ssr	^0.10.3	runtime
@supabase/supabase-js	^2.106.0	runtime
@vercel/analytics	^2.0.1	runtime
@vercel/speed-insights	^2.0.0	runtime
next	^16.2.4	runtime
pino	^10.3.1	runtime
react	^19.2.5	runtime
react-dom	^19.2.5	runtime
resend	^6.14.0	runtime
@eslint/js	^10.0.1	dev/tooling
@opennextjs/cloudflare	^1.20.1	dev/tooling
@testing-library/react	^16.3.2	dev/tooling
@testing-library/user-event	^14.6.1	dev/tooling
@types/node	^26.0.0	dev/tooling
@types/react	^19.2.17	dev/tooling
@types/react-dom	^19.2.3	dev/tooling
eslint	^10.4.1	dev/tooling
eslint-config-prettier	^10.1.8	dev/tooling
globals	^17.6.0	dev/tooling
jsdom	^29.1.1	dev/tooling
prettier	^3.8.3	dev/tooling
sharp	^0.35.1	dev/tooling
supabase	^2.100.1	dev/tooling
typescript	^6.0.3	dev/tooling
vitest	^4.1.9	dev/tooling
wrangler	^4.107.1	dev/tooling

As versoes acima vem de package.json, que e a fonte de verdade do projeto. O diretorio node_modules local pode estar inconsistente em maquinas de desenvolvimento e nao deve ser usado como inventario tecnico.

2. Next.js App Router

Next.js organiza a aplicacao por arquivos dentro de app/. Cada pasta vira uma rota. Arquivos page.js renderizam paginas; route.js implementam endpoints; loading.js define estado de carregamento; layout.js envolve todas as paginas.

app/layout.js - layout raiz e Speed Insights

```
 1 | import "./globals.css";
 2 |
 3 | import { Analytics } from "@vercel/analytics/next";
 4 | import { SpeedInsights } from "@vercel/speed-insights/next";
 5 |
 6 | import { AuthHashBridge } from "@/src/auth/auth-hash-bridge.js";
 7 | import { NavigationLoadingOverlay } from
   "@/src/components/loading/navigation-loading-overlay.js";
 8 | import { SiteFooter } from "@/src/components/site-footer.js";
 9 |
10 | export const metadata = {
11 |   metadataBase: new URL("https://www.tszr15-store.com.br"),
12 |   title: "TSZR15 | Loja R15 com conta de cliente",
13 |   description:
14 |     "Catalogo Yamaha R15 com busca, conta de cliente, dados de entrega e fechamento via
   WhatsApp Business.",
15 |   icons: {
16 |     icon: "/brand/logo-tszr15-store.png"
17 |   }
18 | };
19 |
20 | export default function RootLayout({ children }) {
21 |   return (
22 |     <html data-scroll-behavior="smooth" lang="pt-BR">
23 |       <body>
24 |         <AuthHashBridge />
```

O layout raiz configura idioma, metadados, bridge de autenticacao, overlay de navegacao e o componente SpeedInsights da Vercel. Isso faz com que toda rota compartilhe a mesma base visual e de comportamento.

app/page.js - Server Component com revalidacao

```
 1 | import globalStyles from "@/app/storefront.module.css";
 2 | import { cx } from "@/src/lib/classnames";
 3 | import { CatalogHub } from "@/src/components/catalog/catalog-hub.js";
 4 | import { getStorefrontMenu } from "@/src/catalog/index.js";
 5 | import { getPublicCatalogProductsForStorefront } from
   "@/src/catalog/supabase-catalog.js";
 6 |
 7 | // As ações administrativas já invalidam estas rotas; o TTL curto cobre alterações
   externas.
 8 | export const revalidate = 60;
 9 |
10 | export default async function HomePage() {
11 |   const catalog = await getPublicCatalogProductsForStorefront();
12 |   const menu = getStorefrontMenu(catalog.products);
13 |
14 |   return (
15 |     <main className={cx(globalStyles, "page-shell")}>
```

A home e async, roda no servidor e usa revalidate = 3600 para cache incremental. A pagina busca produtos publicados, monta o menu e entrega os dados prontos ao componente cliente CatalogHub.

app/api/catalog/route.js - API route estatica

```
import { getStorefrontMenu } from "@/src/catalog/index.js";
import { getPublicCatalogProductsForStorefront } from
"@/src/catalog/supabase-catalog.js";

export const dynamic = "force-static";
export const revalidate = 60;

export async function GET() {
  const catalog = await getPublicCatalogProductsForStorefront();

  return Response.json({
    categories: getStorefrontMenu(catalog.products),
    products: catalog.products,
    source: catalog.source
  });
}
```

API routes do App Router exportam funcoes HTTP como GET e POST. Aqui o catalogo publico vira JSON, com dynamic = force-static e revalidate = 3600 para evitar consulta desnecessaria a cada request.

Rotas importantes

Rota	Arquivo	Papel
/	app/page.js	Vitrine principal com catalogo e busca.
/catalogo	app/catalogo/page.js	Entrada dedicada para produtos.
/produto/[slug]	app/produto/[slug]/page.js	Detalhe do produto, imagens, avaliacoes e relacionados.
/pedido	app/pedido/page.js	Carrinho e checkout.
/conta	app/conta/page.js + actions.js	Perfil, enderecos, pedidos e avaliacoes.
/admin	app/admin/page.js + actions.js	Operacao interna, catalogo, cupons, pedidos e moderacao.
/rastreio	app/rastreio/page.js	Consulta publica por pedido e contato.
/api/checkout/whatsapp	app/api/checkout/whatsapp/route.js	Valida e persiste pedido, devolve URL do WhatsApp.

3. React no cliente

Componentes que precisam de estado, eventos do navegador, localStorage ou chamadas fetch no browser usam a diretiva 'use client'. O servidor entrega dados iniciais; React controla interacao fina da vitrine e do checkout.

src/components/catalog/catalog-hub.js - busca, categoria ativa e rendering

```
export function CatalogHub({ categories, currentUser, products }) {
  const [activeCategory, setActiveCategory] = useState("all");
  const [query, setQuery] = useState("");
  const deferredQuery = useDeferredValue(query);
  const normalizedQuery = normalizeSearch(deferredQuery);
  const featuredProducts = useMemo(() => getFeaturedProducts(products), [products]);

  const matchingProducts = products.filter((product) => {
    const searchable = normalizeSearch(`${product.name} ${product.productFamily}`);
    const matchesQuery = normalizedQuery.length === 0 ||
searchable.includes(normalizedQuery);

    return matchesQuery;
  });
  const visibleProducts = matchingProducts.filter(
    (product) => activeCategory === "all" ||
product.storefrontCategoryIds.includes(activeCategory)
  );
  const visibleCategories = groupProductsByCategory(visibleProducts).filter(
    (category) => category.products.length > 0
  );

  function setSearchValue(value) {
    startTransition(() => setQuery(value));
  }

  return (
    <>
      <StoreHeader currentUser={currentUser} onSearchChange={setSearchValue}
query={query} />

      <section className={cx(globalStyles, "brand-hero")}>
        <div className={cx(globalStyles, "brand-hero-copy")}>
          <div className={cx(globalStyles, "hero-brand-row")}>
            <Image
              alt="TSZ Store"
              className={cx(globalStyles, "hero-logo")}
              height={2000}
              sizes="188px"
              src={brandLogoSrc}
              width={2000}
            />
            <p className={cx(globalStyles, "hero-kicker")}>Performance parts for Yamaha
R15</p>
          </div>
          <h1>
            Sua R15 <span>em outro nível</span>
          </h1>
          <p className={cx(globalStyles, "hero-lead")}>
```

useDeferredValue reduz pressao da busca; startTransition evita travar a UI ao trocar filtros.

src/components/catalog/cart-checkout.js - estados do carrinho

```
// marcador nao encontrado em src/components/catalog/cart-checkout.js: const [cartItems
```

O checkout controla carrinho, cupom, frete, consentimento, CEP e status de envio no estado React.

src/components/catalog/catalog-shared.js - localStorage do carrinho

```
136 | export function readStoredCart() {
137 |   if (typeof window === "undefined") {
138 |     return [];
139 |   }
140 |
141 |   try {
142 |     const storedValue = window.localStorage.getItem(cartStorageKey);
143 |     const parsedItems = storedValue ? JSON.parse(storedValue) : [];
144 |
145 |     return Array.isArray(parsedItems) ? parsedItems : [];
146 |   } catch {
147 |     return [];
148 |   }
149 | }
150 |
151 | export function writeStoredCart(items) {
152 |   if (typeof window === "undefined") {
153 |     return;
154 |   }
155 |
156 |   if (!Array.isArray(items) || items.length === 0) {
157 |     window.localStorage.removeItem(cartStorageKey);
158 |   } else {
159 |     window.localStorage.setItem(cartStorageKey, JSON.stringify(items));
160 |   }
161 |
162 |   window.dispatchEvent(new Event("tszr15-cart-changed"));
163 | }
164 |
165 | export function clearStoredCart() {
166 |   if (typeof window === "undefined") {
167 |     return;
168 |   }
169 |
170 |   window.localStorage.removeItem(cartStorageKey);
```

O carrinho fica no browser e dispara um evento customizado para atualizar badges sem recarregar a pagina.

Regra pratica

Se o codigo usa cookies(), headers(), Supabase service_role ou acesso sensivel, ele fica no servidor. Se usa localStorage, window, eventos e inputs interativos, ele fica em componente cliente.

4. Supabase: Auth, dados e storage

Supabase aparece como a plataforma de banco, autenticacao, storage e RPC. O projeto separa tres clientes: browser, server SSR com cookies e service_role para operacoes administrativas ou de convidado no servidor.

src/lib/supabase/config.js - leitura de envs e preview

```
const SUPABASE_URL_ENV_KEYS = ["NEXT_PUBLIC_SUPABASE_URL", "SUPABASE_URL"];
const SUPABASE_PREVIEW_URL_ENV_KEYS = ["NEXT_PUBLIC_SUPABASE_PREVIEW_URL",
  "SUPABASE_PREVIEW_URL"];
const SUPABASE_PUBLISHABLE_KEY_ENV_KEYS = [
  "NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY",
  "SUPABASE_PUBLISHABLE_KEY",
  "NEXT_PUBLIC_SUPABASE_ANON_KEY",
  "SUPABASE_ANON_KEY"
];
const SUPABASE_PREVIEW_PUBLISHABLE_KEY_ENV_KEYS = [
  "NEXT_PUBLIC_SUPABASE_PREVIEW_PUBLISHABLE_KEY",
  "SUPABASE_PREVIEW_PUBLISHABLE_KEY",
  "NEXT_PUBLIC_SUPABASE_PREVIEW_ANON_KEY",
  "SUPABASE_PREVIEW_ANON_KEY"
];
const SUPABASE_SERVICE_ROLE_KEY_ENV_KEYS = ["SUPABASE_SERVICE_ROLE_KEY",
"SUPABASE_SECRET_KEY"];
const SUPABASE_PREVIEW_SERVICE_ROLE_KEY_ENV_KEYS = [
  "SUPABASE_PREVIEW_SERVICE_ROLE_KEY",
  "SUPABASE_PREVIEW_SECRET_KEY"
];

function isPreviewRuntime() {
  const explicitTarget = String(
    process.env.SUPABASE_RUNTIME_TARGET ??
process.env.NEXT_PUBLIC_SUPABASE_RUNTIME_TARGET ?? ""
  ).toLowerCase();

  if (explicitTarget) {
    return explicitTarget === "preview";
  }

  return process.env.VERCEL_ENV === "preview";
}

function firstConfiguredEnvValue(keys) {
  return keys.map((key) => process.env[key]).find(Boolean) ?? "";
}

function envKeysForRuntime(previewKeys, fallbackKeys) {
  return isPreviewRuntime() ? [...previewKeys, ...fallbackKeys] : fallbackKeys;
}

export function getSupabaseUrl() {
  return firstConfiguredEnvValue(
    envKeysForRuntime(SUPABASE_PREVIEW_URL_ENV_KEYS, SUPABASE_URL_ENV_KEYS)
  );
}

export function getSupabasePublishableKey() {
  return firstConfiguredEnvValue(
    envKeysForRuntime(SUPABASE_PREVIEW_PUBLISHABLE_KEY_ENV_KEYS,
SUPABASE_PUBLISHABLE_KEY_ENV_KEYS)
  );
}

export function getSupabaseServiceRoleKey() {
  return firstConfiguredEnvValue(
```

src/lib/supabase/server.js - cliente SSR com cookies

```
import { cookies } from "next/headers";
import { createServerClient } from "@supabase/ssr";

import { getPublicSupabaseConfig } from "./config.js";

export async function createServerSupabaseClient() {
  const config = getPublicSupabaseConfig();

  if (!config.isConfigured) {
    return null;
  }

  const cookieStore = await cookies();

  return createServerClient(config.url, config.publishableKey, {
    cookies: {
      getAll() {
        return cookieStore.getAll();
      },
      setAll(cookiesToSet) {
        try {
          cookiesToSet.forEach(({ name, value, options }) => {
            cookieStore.set(name, value, options);
          });
        } catch {
          // Server Components cannot write cookies. The root proxy refreshes sessions.
        }
```

src/lib/supabase/client.js - cliente do navegador

```
import { createBrowserClient } from "@supabase/ssr";

import { getPublicSupabaseConfig } from "./config.js";

export function createBrowserSupabaseClient() {
  const config = getPublicSupabaseConfig();

  if (!config.isConfigured) {
    return null;
  }

  return createBrowserClient(config.url, config.publishableKey);
}
```

src/lib/supabase/admin.js - service_role somente no servidor

```
import "server-only";

import { createClient } from "@supabase/supabase-js";

import { getSupabaseServiceRoleKey, getSupabaseUrl } from "./config.js";

export function createServiceRoleSupabaseClient() {
  const url = getSupabaseUrl();
  const serviceRoleKey = getSupabaseServiceRoleKey();

  if (!url || !serviceRoleKey) {
    return null;
  }

  return createClient(url, serviceRoleKey, {
    auth: {
      autoRefreshToken: false,
      persistSession: false
    }
  });
}
```

O cliente service_role importa 'server-only'. Isso evita que codigo privilegiado seja empacotado para o navegador. Quando a chave nao existe, as funcoes retornam null e o app reduz funcionalidade em vez de expor segredo.

middleware.js - refresh de sessao e protecao de /admin

```
export async function middleware(request) {
  const supabaseUrl = getSupabaseUrl();
  const supabaseKey = getSupabasePublishableKey();
  const isAdminRequest = isAdminPath(request.nextUrl.pathname);

  let response = NextResponse.next({ request });

  if (isAdminRequest) {
    const adminSessionValue = request.cookies.get(ADMIN_SESSION_COOKIE)?.value ?? "";
    const hasSignedAdminSession = await isAdminSessionValueValidAtEdge(adminSessionValue);
    const hasFreshAdminSessionShape = isAdminSessionValueFreshShapeAtEdge(adminSessionValue);

    if (!hasSignedAdminSession && !hasFreshAdminSessionShape) {
      return redirectToAdminLogin(request);
    }

    return addAdminSecurityHeaders(response);
  }

  if (!supabaseUrl || !supabaseKey) {
    return response;
  }

  const supabase = createServerClient(supabaseUrl, supabaseKey, {
    cookies: {
      getAll() {
        return request.cookies.getAll();
      },
      setAll(cookiesToSet) {
        cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value));
        response = NextResponse.next({ request });
        cookiesToSet.forEach(({ name, value, options }) => {
          response.cookies.set(name, value, options);
        });
      }
    }
  });

  await supabase.auth.getUser();

  return response;
}

export const config = {
  matcher: [
    "/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)"
  ]
```

O proxy atualiza cookies de sessao Supabase fora de /api e aplica headers no-store/noindex para area administrativa.

Banco Postgres, RLS e RPC

Area	Tabelas	Origem
Cliente	public.customer_profiles, public.customer_addresses, public.assisted_purchase_consents	Definido em supabase/migrations
Pedido	public.orders, public.order_items, public.payments, public.audit_logs	Definido em supabase/migrations
Operacao	public.supplier_purchases, public.supplier_tracking_events, public.support_threads	Definido em supabase/migrations
Catalogo	public.catalog_categories, public.catalog_products, public.catalog_product_categories, public.catalog_product_costs, public.catalog_coupons	Definido em supabase/migrations
Avaliacoes	public.order_item_reviews, public.order_review_photos	Definido em supabase/migrations
Seguranca	private.api_rate_limits	Definido em supabase/migrations

catalog_products - tabela de catalogo publicada

```
11 | create table if not exists public.catalog_products (
12 |   id text primary key,
13 |   slug text not null unique,
14 |   name text not null,
15 |   storefront_category_ids text[] not null default '{}',
16 |   product_family text not null,
17 |   bike_model_scope text[] not null default '{}',
18 |   price_cents integer not null check (price_cents > 0),
19 |   currency text not null default 'BRL',
20 |   variations text[] not null default '{}',
21 |   availability text not null default 'sob-consulta',
22 |   lead_time_days integer not null default 2 check (lead_time_days >= 0),
23 |   shipping_class text not null default 'medium',
24 |   checkout_channel text not null default 'whatsapp-business',
25 |   internal_purchase_source jsonb not null default '{}'::jsonb,
26 |   notes text not null default '',
27 |   is_published boolean not null default true,
28 |   created_at timestamptz not null default now(),
29 |   updated_at timestamptz not null default now(),
30 |   constraint catalog_products_currency_check check (currency = 'BRL'),
31 |   constraint catalog_products_checkout_channel_check check (checkout_channel =
    'whatsapp-business'),
32 |   constraint catalog_products_storefront_categories_check check
    (array_length(storefront_category_ids, 1) >= 1),
33 |   constraint catalog_products_variations_check check (array_length(variations, 1) >= 1)
34 | );
35 |
```

orders - pedido com status, snapshots e canal de atendimento

```
171 | create table if not exists public.orders (
172 |   id uuid primary key default gen_random_uuid(),
173 |   order_number text not null unique default public.generate_order_number(),
174 |   user_id uuid references auth.users(id) on delete set null,
175 |   customer_name text not null,
176 |   customer_email text,
177 |   customer_whatsapp text,
178 |   customer_phone text,
179 |   customer_tax_id text,
180 |   customer_snapshot jsonb not null default '{}'::jsonb,
181 |   address_snapshot jsonb not null default '{}'::jsonb,
182 |   subtotal_cents integer not null default 0 check (subtotal_cents >= 0),
183 |   discount_cents integer not null default 0 check (discount_cents >= 0),
184 |   shipping_cents integer not null default 0 check (shipping_cents >= 0),
185 |   total_cents integer not null default 0 check (total_cents >= 0),
186 |   currency text not null default 'BRL',
187 |   shipping_option_id text not null,
188 |   shipping_label text not null,
189 |   shipping_eta text,
190 |   payment_method_id text not null,
191 |   payment_method_label text not null,
192 |   payment_status text not null default 'aguardando_pagamento'
193 |     check (payment_status in (
194 |       'aguardando_pagamento',
195 |       'pagamento_confirmado',
196 |       'cancelado',
197 |       'reembolsado'
198 |     )),
199 |   operational_status text not null default 'enviado_whatsapp_business'
200 |     check (operational_status in (
201 |       'orcamento_iniciado',
202 |       'enviado_whatsapp_business',
203 |       'aguardando_atendimento',
204 |       'dados_incompletos',
205 |       'aguardando_pagamento',
206 |       'pagamento_confirmado',
207 |       'origem_interna_em_validacao',
208 |       'compra_interna_pendente',
209 |       'compra_interna_realizada',
210 |       'aguardando_postagem_envio',
211 |       'rastreio_recebido',
212 |       'em_transito',
```

create_checkout_order - RPC transacional do checkout

```
450 | create or replace function public.create_checkout_order(
451 |   p_user_id uuid,
452 |   p_customer_snapshot jsonb,
453 |   p_address_snapshot jsonb,
454 |   p_items jsonb,
455 |   p_totals jsonb,
456 |   p_payment jsonb,
457 |   p_shipping jsonb,
458 |   p_message text,
459 |   p_consent_snapshot jsonb,
460 |   p_request_context jsonb default '{}'::jsonb
461 | )
462 | returns jsonb
463 | language plpgsql
464 | as $$
465 | declare
466 |   v_auth_user_id uuid := auth.uid();
467 |   v_consent_id uuid := null;
468 |   v_item jsonb;
469 |   v_order_id uuid;
470 |   v_order_number text;
471 | begin
472 |   if p_user_id is null and v_auth_user_id is not null then
473 |     p_user_id := v_auth_user_id;
474 |   end if;
475 |
476 |   if p_user_id is not null and v_auth_user_id is not null and p_user_id <>
    v_auth_user_id then
477 |     raise exception 'Nao e permitido criar pedido para outro usuario.';
478 |   end if;
479 |
480 |   if jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0 then
481 |     raise exception 'Pedido sem itens.';
482 |   end if;
483 |
484 |   if p_user_id is not null and coalesce((p_consent_snapshot ->> 'accepted')::boolean,
    false) then
485 |     insert into public.assisted_purchase_consents (
486 |       user_id,
487 |       consent_version,
488 |       consent_text,
489 |       purchase_data_use_authorized,
490 |       ip_address,
491 |       user_agent
492 |     )
493 |     values (
494 |       p_user_id,
```

A API chama esta funcao para gravar pedido, itens, pagamento e auditoria dentro do banco.

RLS

As migrations habilitam Row Level Security nas tabelas publicas. Cliente autenticado so ve/insere seus proprios perfis, enderecos, pedidos e avaliacoes. Dados internos de fornecedor e custos ficam para service_role/admin.

Storage buckets de imagens

```
72 | insert into storage.buckets (
73 |   id,
74 |   name,
75 |   public,
76 |   file_size_limit,
77 |   allowed_mime_types
78 | ) values (
79 |   'product-images',
80 |   'product-images',
81 |   true,
82 |   5242880,
83 |   array['image/jpeg', 'image/png', 'image/webp', 'image/gif']::text[]
84 | )
85 | on conflict (id) do update set
86 |   public = excluded.public,
87 |   file_size_limit = excluded.file_size_limit,
88 |   allowed_mime_types = excluded.allowed_mime_types;
89 |
90 | alter table public.orders
91 |   add column if not exists discount_snapshot jsonb not null default '{}'::jsonb;
92 |
93 | alter table public.order_items
94 |   add column if not exists unit_cost_cents integer check (unit_cost_cents is null or
    unit_cost_cents >= 0),
95 |   add column if not exists subtotal_cost_cents integer check (subtotal_cost_cents is
    null or subtotal_cost_cents >= 0);
96 |
97 | create or replace function public.redeem_catalog_coupon(p_code text)
98 | returns void
99 | language plpgsql
```

O projeto usa buckets publicos/controlados para imagens de produtos e fotos de avaliacoes.

5. Catalogo e vitrine

O catalogo nasce no Supabase, mas a aplicacao converte linhas SQL em objetos de dominio JavaScript. Antes de chegar ao cliente, metadados internos como custo, margem e origem de compra sao removidos.

src/catalog/supabase-catalog-core.js - leitura do catalogo publicado

```
export function toCatalogProduct(row) {
  return {
    id: row.id,
    slug: row.slug,
    name: row.name,
    storefrontCategoryIds: row.storefront_category_ids ?? [],
    productFamily: row.product_family,
    bikeModelScope: row.bike_model_scope ?? [],
    priceCents: row.price_cents,
    currency: row.currency,
    variations: row.variations ?? [],
    availability: row.availability,
    leadTimeDays: row.lead_time_days,
    shippingClass: row.shipping_class,
    imageUrls: row.image_urls ?? [],
    checkoutChannel: row.checkout_channel,
    internalPurchaseSource: row.internal_purchase_source ?? {},
    notes: row.notes ?? ""
  };
}

const publicCatalogProductColumns = `
  id,
  slug,
  name,
  storefront_category_ids,
  product_family,
  bike_model_scope,
  price_cents,
  currency,
  variations,
  availability,
  lead_time_days,
  shipping_class,
```

src/catalog/index.js - remocao de metadados internos

```
    ...category,
    productCount: products.filter((product) =>
product.storefrontCategoryIds.includes(category.id))
      .length
  }));
}

export function toPublicCatalogProduct(product) {
  const {
    costCents: _costCents,
    internalPurchaseCandidates: _internalPurchaseCandidates,
    internalPurchaseSource: _internalPurchaseSource,
    marginPercent: _marginPercent,
    profitCents: _profitCents,
    supplierSource: _supplierSource,
    ...publicProduct
  } = product;

```

src/admin/catalog-admin.js - produto criado pelo admin

```
272 | function collectProductPayload(formData) {
273 |   const name = cleanString(formData.get("name"), 180);
274 |   const previousId = cleanString(formData.get("productId"), 160);
275 |   const slug = slugify(formData.get("slug") || name);
276 |   const id = previousId || slug;
277 |   const storefrontCategoryIds = getSelectedCategoryIds(formData);
278 |   const productFamily = cleanString(formData.get("productFamily"), 80);
279 |   const priceCents = parseMoneyToCents(formData.get("price"));
280 |   const costCents = parseOptionalMoneyToCents(formData.get("cost"));
281 |   const variations = splitList(formData.get("variations"), { maxItems: 24, maxLength:
    120 });
282 |   const imageUrls = splitImageUrls(formData.get("imageUrls"));
283 |   const bikeModelScope = splitList(formData.get("bikeModelScope"), {
284 |     maxItems: 8,
285 |     maxLength: 80
286 |   });
287 |
288 |   if (!name) {
289 |     throw new Error("Informe o nome do produto.");
290 |   }
291 |
292 |   if (!id || !slug) {
293 |     throw new Error("Informe um slug/ID valido para o produto.");
294 |   }
295 |
296 |   if (storefrontCategoryIds.length === 0) {
297 |     throw new Error("Selecione pelo menos uma categoria.");
298 |   }
299 |
300 |   if (!technicalFamilies.includes(productFamily)) {
301 |     throw new Error("Familia tecnica invalida.");
302 |   }
303 |
304 |   if (!Number.isInteger(priceCents)) {
305 |     throw new Error("Informe um preco do cliente valido.");
306 |   }
307 |
308 |   if (cleanString(formData.get("cost"), 40) && !Number.isInteger(costCents)) {
309 |     throw new Error("Informe um preco real valido ou deixe vazio.");
310 |   }
311 |
312 |   if (variations.length === 0) {
313 |     throw new Error("Informe pelo menos uma variacao.");
314 |   }
315 |
316 |   const invalidImageUrl = imageUrls.find((imageUrl) => !isValidImageUrl(imageUrl));
317 |
318 |   if (invalidImageUrl) {
319 |     throw new Error(`URL de imagem invalida: ${invalidImageUrl}`);
320 |   }
321 |
```

O admin cria/edita produtos, custos, variacoes, categorias e URLs de imagens; depois revalida as rotas estaticas.

6. Checkout, WhatsApp e pedidos

O checkout e desenhado para nao confiar no navegador. O cliente envia o carrinho, mas o backend recarrega o catalogo, valida produtos/variacoes, recalcula precos e so entao monta o pedido e a mensagem.

app/api/checkout/whatsapp/route.js - ponto de entrada do checkout

```
 63 | export async function POST(request) {
 64 |   const serviceSupabase = createServiceRoleSupabaseClient();
 65 |   const rateLimit = await consumeRateLimit({
 66 |     ...rateLimitProfiles.checkout,
 67 |     identifier: getRequestIp(request),
 68 |     supabase: serviceSupabase
 69 |   });
 70 |
 71 |   if (!rateLimit.allowed) {
 72 |     logServerEvent("warn", "checkout_rate_limit_blocked", {
 73 |       retryAfterSeconds: rateLimit.retryAfterSeconds,
 74 |       unavailable: rateLimit.unavailable
 75 |     });
 76 |
 77 |     return createRateLimitResponse(rateLimit, {
 78 |       rateLimitedMessage: "Muitas tentativas de checkout. Tente novamente em
     instantes."
 79 |     });
 80 |   }
 81 |
 82 |   let payload;
 83 |
 84 |   try {
 85 |     payload = await request.json();
 86 |   } catch {
 87 |     logServerEvent("warn", "checkout_validation_error", {
 88 |       reason: "invalid_json"
 89 |     });
 90 |     return errorResponse("Envie um JSON valido para finalizar o pedido.", 400);
 91 |   }
 92 |
 93 |   const storeName = process.env.NEXT_PUBLIC_STORE_NAME ?? "TSZR15";
 94 |   const phoneNumber =
 95 |     process.env.WHATSAPP_BUSINESS_NUMBER ??
 96 |     process.env.NEXT_PUBLIC_WHATSAPP_BUSINESS_NUMBER ??
 97 |     "5511999999999";
 98 |   const userSupabase = await createServerSupabaseClient();
 99 |   const supabase = serviceSupabase ?? userSupabase;
100 |
101 |   let draft;
102 |
103 |   try {
104 |     const catalog = await getSupabaseCatalogProducts();
105 |     const products = await attachInternalProductCosts(catalog.products,
    serviceSupabase);
106 |     const baseDraft = buildCheckoutOrderDraft(payload, { products, storeName });
107 |     const coupon = await resolveCheckoutCoupon({
108 |       cartItems: baseDraft.cartItems,
109 |       code: payload?.couponCode,
110 |       supabase: serviceSupabase
111 |     });
112 |
113 |     draft = coupon ? buildCheckoutOrderDraft(payload, { coupon, products, storeName })
    : baseDraft;
114 |
115 |     const unavailableItems = draft.cartItems.filter((item) => {
116 |       const stock = getVariationStockStatus(
117 |         products.find((product) => product.id === item.id),
118 |         item.variation
119 |       );
120 |
121 |       return !stock.canAddToCart || (stock.quantity !== null && item.quantity >
    stock.quantity);
122 |     });
123 |
124 |     if (unavailableItems.length > 0) {
125 |       return errorResponse(
126 |         "Uma ou mais variações ficaram esgotadas antes de finalizar o pedido.",
127 |         409,
128 |         unavailableItems.map((item) => `${item.name} (${item.variation})`)
129 |       );
```

A rota aplica rate limit, parseia JSON, pega telefone da loja, cria cliente Supabase e monta o draft do pedido.

src/checkout/order-backend.js - persistencia via RPC

```
export async function persistCheckoutOrder({ draft, requestContext = {}, supabase, user
}) {
  if (!supabase) {
    return {
      reason: "Supabase nao configurado.",
      saved: false
    };
  }

  if (!user && !getSupabaseServiceRoleKey()) {
    return {
      reason: "Pedido de convidado precisa de SUPABASE_SERVICE_ROLE_KEY no servidor.",
      saved: false
    };
  }

  const { data, error } = await supabase.rpc("create_checkout_order", {
    p_address_snapshot: draft.addressSnapshot,
    p_consent_snapshot: draft.consentSnapshot,
    p_customer_snapshot: draft.customerSnapshot,
    p_items: draft.databaseItems,
    p_message: draft.message,
    p_payment: draft.payment,
    p_request_context: requestContext,
    p_shipping: draft.shipping,
    p_totals: draft.totals,
    p_user_id: user?.id ?? null
  });

  if (error) {
    throw new CheckoutPersistenceError(error.message);
  }

  return {
    id: data?.id ?? null,
    operationalStatus: data?.operationalStatus ?? "enviado_whatsapp_business",
    orderNumber: data?.orderNumber ?? null,
    paymentStatus: data?.paymentStatus ?? "aguardando_pagamento",
    saved: true
```

src/checkout/whatsapp.js - mensagem e URL wa.me

```
export function buildWhatsAppCheckoutUrl({ phoneNumber, message }) {
  const normalizedPhoneNumber = normalizePhoneNumber(phoneNumber);

  if (!normalizedPhoneNumber) {
    return "";
  }

  return `https://wa.me/${normalizedPhoneNumber}?text=${encodeURIComponent(message)}`;
}
```

encodeURIComponent evita quebrar a URL com quebras de linha, acentos e simbolos monetarios.

src/components/catalog/cart-checkout.js - envio pelo browser

```
// marcador nao encontrado em src/components/catalog/cart-checkout.js: async function
    submitCheckout
```

Depois que a API responde, o browser limpa o carrinho e abre o WhatsApp em nova aba.

7. Cupons, rate limit e logs

Cupons e checkout usam protecoes de abuso. O identificador de rate limit e hasheado antes de ir ao banco, e em producao o sistema falha fechado se o Supabase de rate limit estiver indisponivel.

src/checkout/coupons.js - normalizacao e regras do cupom

```
import { CheckoutValidationError } from "./order-backend.js";

export function normalizeCouponCode(value) {
  return String(value ?? "")
    .trim()
    .toUpperCase()
    .replace(/[^A-Z0-9_-]/g, "")
    .slice(0, 40);
}

function cents(value) {
  return Number.isInteger(value) && value > 0 ? value : 0;
}

function hasIntersection(left = [], right = []) {
  const rightSet = new Set(right);
  return left.some((value) => rightSet.has(value));
}

function getDiscountableSubtotal(cartItems, coupon) {
  const productIds = coupon.appliesToProductIds ?? [];
  const categoryIds = coupon.appliesToCategoryIds ?? [];
  const isGlobalCoupon = productIds.length === 0 && categoryIds.length === 0;

  return cartItems.reduce((total, item) => {
    const itemSubtotal = item.priceCents * item.quantity;

    if (isGlobalCoupon) {
      return total + itemSubtotal;
    }

    if (productIds.includes(item.id)) {
      return total + itemSubtotal;
    }

    if (hasIntersection(item.storefrontCategoryIds, categoryIds)) {
      return total + itemSubtotal;
    }

    return total;
  }, 0);
}

export function toCheckoutCoupon(row, cartItems, now = new Date()) {
  const code = normalizeCouponCode(row?.code);

  if (!code) {
    throw new CheckoutValidationError("Cupom invalido.");
  }

  if (row.is_active === false) {
    throw new CheckoutValidationError("Cupom inativo.");
  }

  if (row.starts_at && new Date(row.starts_at) > now) {
    throw new CheckoutValidationError("Cupom ainda nao esta ativo.");
  }

  if (row.expires_at && new Date(row.expires_at) < now) {
    throw new CheckoutValidationError("Cupom expirado.");
  }

  if (
    Number.isInteger(row.max_redemptions) &&
    Number.isInteger(row.redemption_count) &&
    row.redemption_count >= row.max_redemptions
  ) {
```

src/lib/rate-limit.js - perfis de limitacao

```
const localRateLimitStore = new Map();
const productionEnvironments = new Set(["production", "preview"]);

export const rateLimitProfiles = {
  adminLogin: {
    blockSeconds: 15 * 60,
    limit: 5,
    scope: "admin-login",
    windowSeconds: 15 * 60
  },
  checkout: {
    blockSeconds: 60,
    limit: 10,
    scope: "checkout-whatsapp",
    windowSeconds: 60
  },
  coupon: {
    blockSeconds: 60,
    limit: 20,
    scope: "coupon-validate",
    windowSeconds: 60
  }
};

function isStrictRuntime() {
  return (
    process.env.NODE_ENV === "production" ||
    productionEnvironments.has(process.env.VERCEL_ENV ?? "")
  );
```

supabase/migrations/...api_rate_limits.sql - tabela privada e RPC

```
create schema if not exists private;

create table if not exists private.api_rate_limits (
  scope text not null,
  identifier_hash text not null,
  window_started_at timestamptz not null default now(),
  attempt_count integer not null default 0 check (attempt_count >= 0),
  blocked_until timestamptz,
  updated_at timestamptz not null default now(),
  primary key (scope, identifier_hash)
);

alter table private.api_rate_limits enable row level security;

revoke all on schema private from public, anon, authenticated;
grant usage on schema private to service_role;

revoke all on table private.api_rate_limits from public, anon, authenticated;
grant select, insert, update, delete on table private.api_rate_limits to service_role;

create index if not exists api_rate_limits_updated_at_idx
  on private.api_rate_limits (updated_at);

create or replace function public.consume_rate_limit(
  p_scope text,
  p_identifier_hash text,
  p_limit integer,
  p_window_seconds integer,
  p_block_seconds integer default null,
  p_increment boolean default true
)
returns jsonb
language plpgsql
set search_path = ''
as $$
declare
  v_now timestamptz := now();
  v_row private.api_rate_limits%rowtype;
```

src/lib/logger.js - redacao de dados sensiveis

```
import pino from "pino";

const censor = "[redacted]";
const sensitiveKeyPattern =
  /authorization|cookie|token|password|email|phone|whatsapp|tax|cpf|cnpj|customer|payload|message|url/i;

const redactPaths = [
  "authorization",
  "cookie",
  "headers.authorization",
  "headers.cookie",
  "req.headers.authorization",
  "req.headers.cookie",
  "token",
  "password",
  "email",
  "phone",
  "phoneNumber",
  "whatsapp",
  "whatsappUrl",
  "taxId",
  "tax_id",
  "cpf",
  "cnpj",
  "customer",
  "payload",
  "payload.customer",
  "*.authorization",
  "*.cookie",
  "*.token",
  "*.password",
  "*.email",
  "*.phone",
  "*.phoneNumber",
  "*.whatsapp",
  "*.whatsappUrl",
  "*.taxId",
  "*.tax_id",
  "*.cpf",
  "*.cnpj",
  "*.customer",
  "*.payload"
];

function createBaseLogger() {
  return pino({
    base: undefined,
    level: process.env.LOG_LEVEL ?? "info",
    redact: {
      censor,
      paths: redactPaths,
      remove: false
    }
  });
}

export const logger = createBaseLogger();

```

8. Autenticacao, conta e admin

A autenticacao de cliente usa Supabase Auth. A area admin nao usa conta Supabase normal: ela usa um token TSZR15_ADMIN_TOKEN, valida por hash em tempo constante e cria um cookie assinado com HMAC.

app/auth/actions.js - Server Actions de login/cadastro

```
export async function signInAction(formData) {
  const email = formValue(formData, "email");
  const password = formValue(formData, "password");
  const nextPath = getSafeAuthRedirectPath(formValue(formData, "next"), "/conta");

  if (email.toLowerCase() === "admin") {
    const rateLimitContext = await checkAdminLoginRateLimit();

    if (!rateLimitContext.result.allowed) {
      const message = rateLimitContext.result.unavailable
        ? "Protecao administrativa indisponivel. Tente novamente em instantes."
        : "Muitas tentativas administrativas. Aguarde alguns minutos antes de tentar de novo.";

      redirectWithError("/entrar", message, "/admin");
    }

    if (!isAdminTokenConfigured()) {
      logServerEvent("error", "admin_login_token_missing");
      redirectWithError(
        "/entrar",
        "Token administrativo ausente no servidor. Configure TSZR15_ADMIN_TOKEN no ambiente de producao.",
        "/admin"
      );
    }

    const isValidAdmin = await startAdminSession(password);

    if (!isValidAdmin) {
      const failedAttempt = await recordFailedAdminLogin(rateLimitContext);

      if (!failedAttempt.allowed) {
        redirectWithError(
          "/entrar",
          "Muitas tentativas administrativas. Aguarde alguns minutos antes de tentar de novo.",
          "/admin"
        );
      }

      redirectWithError("/entrar", "Senha administrativa invalida.", "/admin");
    }

    await resetRateLimit({
      ...rateLimitProfiles.adminLogin,
      identifier: rateLimitContext.identifier,
      supabase: rateLimitContext.supabase
    });
    logServerEvent("info", "admin_login_success");
    redirect("/admin");
  }

  const supabase = await createServerSupabaseClient();

  if (!supabase) {
    const status = getSupabaseConfigStatus();
    const missing = status.missing.length ? ` Faltando: ${status.missing.join("; ")}.` : "";
    redirectWithError(
      "/entrar",
      `Configure as variaveis do Supabase antes de entrar.${missing}`,
      nextPath
    );
  }

```

src/admin/admin-session.js - cookie admin assinado

```
import { createHash, createHmac, timingSafeEqual } from "crypto";

import {
  ADMIN_SESSION_COOKIE,
  ADMIN_SESSION_MAX_AGE_SECONDS,
  ADMIN_SESSION_VERSION
} from "./admin-session-constants.js";

export { ADMIN_SESSION_COOKIE, ADMIN_SESSION_MAX_AGE_SECONDS };

export function getConfiguredAdminToken() {
  return process.env.TSZR15_ADMIN_TOKEN ?? "";
}

export function isAdminTokenValueConfigured(token = getConfiguredAdminToken()) {
  return token.length >= 12;
}

function hashValue(value) {
  return createHash("sha256").update(String(value ?? "")).digest("hex");
}

function signAdminSessionPayload(payload, token) {
  return createHmac("sha256", token).update(payload).digest("hex");
}

function parseAdminSessionValue(sessionValue) {
  const [version, expiresAtValue, signature, ...extra] = String(sessionValue ??
"").split(".");
  const expiresAt = Number(expiresAtValue);

  if (
    extra.length > 0 ||
    version !== ADMIN_SESSION_VERSION ||
    !Number.isSafeInteger(expiresAt) ||
    !/^[a-f0-9]{64}$/i.test(signature ?? "")
  ) {
    return null;
  }

  return {
    expiresAt,
    signature,
    version
  };
}

function safeEqual(left, right) {
  if (!left || !right || left.length !== right.length) {
    return false;
  }

  return timingSafeEqual(Buffer.from(left), Buffer.from(right));
}

```

app/admin/actions.js - Server Action administrativa

```
"use server";

import { headers } from "next/headers";
import { redirect } from "next/navigation";
import { revalidatePath } from "next/cache";

import { clearAdminSession, isAdminSessionValid } from "@/src/admin/admin-auth.js";
import {
  archiveAdminCoupon,
  archiveAdminCatalogProduct,
  upsertAdminCoupon,
  upsertAdminCatalogProduct
} from "@/src/admin/catalog-admin.js";
import {
  createAdminManualOrder,
  setAdminInternalOrderStatus,
  updateAdminOrderOperation
} from "@/src/admin/order-admin.js";
import { moderateOrderReview } from "@/src/reviews/order-reviews.js";
import { revalidateCatalogPaths } from "@/src/catalog/revalidation.js";

function formValue(formData, key) {
  return String(formData.get(key) ?? "").trim();
}

function redirectWithError(path, message) {
  const separator = path.includes("?") ? "&" : "?";
  redirect(`${path}${separator}error=${encodeURIComponent(message)}`);
}

async function isSameOriginAdminRequest() {
  const headerStore = await headers();
  const origin = headerStore.get("origin");
  const host = headerStore.get("host");

  if (!origin || !host) {
    return true;
  }

  try {
    return new URL(origin).host === host;
  } catch {
    return false;
  }
}

export async function adminSignOutAction() {
  if (!(await isSameOriginAdminRequest())) {
    redirect("/entrar?next=/admin");
  }

  await clearAdminSession();
  redirect("/entrar?next=/admin");
}
```

Por que Server Actions?

Formularios de conta e admin podem chamar funcoes do servidor diretamente. Isso permite validar sessao, checar origem, atualizar Supabase e chamar revalidatePath sem expor chaves ao navegador.

9. Rastreio e avaliacoes

Rastreio publico e avaliacoes usam dados internos, mas filtram a exposicao. O cliente consulta por numero do pedido e contato; fotos de avaliacao ficam no Storage e sao servidas por URLs assinadas quando necessario.

src/tracking/order-tracking.js - consulta segura do rastreio

```
export async function findPublicOrderTracking({ contact, orderNumber, supabase }) {
  const client = supabase ?? createServiceRoleSupabaseClient();

  if (!client) {
    return {
      reason: "tracking_not_configured",
      status: "setup-required"
    };
  }

  const cleanOrderNumber = cleanString(orderNumber, 80).toUpperCase();

  if (!cleanOrderNumber || !cleanString(contact, 120)) {
    return {
      reason: "missing-fields",
      status: "empty"
    };
  }

  const { data: order, error } = await client
    .from("orders")
    .select("*")
    .eq("order_number", cleanOrderNumber)
    .limit(1)
    .maybeSingle();

  if (error) {
    throw new Error(error.message);
  }

  if (!order || !contactMatchesOrder(order, contact)) {
    return {
      reason: "not-found",
      status: "not-found"
    };
  }

  const [
    { data: items, error: itemsError },
    { data: supplierPurchases, error: supplierError },
    { data: trackingEvents, error: trackingError }
  ] = await Promise.all([
    client
      .from("order_items")
      .select("product_name, variation, quantity")
      .eq("order_id", order.id)
      .order("created_at"),
    client
      .from("supplier_purchases")
      .select("carrier, source_eta, tracking_code")
      .eq("order_id", order.id)
      .order("created_at")
      .limit(1),
    client
      .from("supplier_tracking_events")
      .select("id, event_status, event_at, location, description, created_at")
      .eq("order_id", order.id)
      .order("event_at", { ascending: false })
      .order("created_at", { ascending: false })
  ]);

  const firstError = itemsError ?? supplierError ?? trackingError;
```

src/reviews/order-reviews.js - upload e assinatura de fotos

```
async function uploadReviewPhotos({ files, reviewId, supabase, userId }) {
  const rows = [];

  for (const [index, file] of files.entries()) {
    const arrayBuffer = await file.arrayBuffer();
    const bytes = new Uint8Array(arrayBuffer);
    const detectedMimeType = detectImageMimeType(bytes);
    const errors = validateReviewImageMeta({
      detectedMimeType,
      sizeBytes: Number(file.size)
    });

    if (file.type && file.type !== detectedMimeType) {
      errors.push("O tipo da foto nao confere com o conteudo do arquivo.");
    }

    if (errors.length > 0) {
      throw new Error(errors[0]);
    }

    const extension = getReviewImageExtension(detectedMimeType);
    const storagePath = `${userId}/${reviewId}/${randomUUID()}.${extension}`;
    const { error: uploadError } = await supabase.storage
      .from(reviewPhotoBucket)
      .upload(storagePath, Buffer.from(arrayBuffer), {
        cacheControl: "3600",
        contentType: detectedMimeType,
        upsert: false
      });

    if (uploadError) {
      throw new Error(`Upload de foto falhou: ${uploadError.message}`);
    }

    rows.push({
      mime_type: detectedMimeType,
      review_id: reviewId,
      size_bytes: Number(file.size),
      sort_order: index,
      storage_bucket: reviewPhotoBucket,
      storage_path: storagePath
    });
  }

  if (rows.length === 0) {
    return [];
  }

  const { error } = await supabase.from("order_review_photos").insert(rows);

  if (error) {
    throw new Error(error.message);
  }

  return rows;
}

async function deleteReviewPhotos({ reviewId, supabase }) {
```

app/rastreio/page.js - pagina publica de consulta

```
export default async function TrackingPage({ searchParams }) {
  const params = await searchParams;
  const orderNumber = params?.pedido ?? "";
  const contact = params?.contato ?? "";
  const hasLookup = Boolean(orderNumber || contact);
  const supabase = await createServerSupabaseClient();
  const snapshot = await getCurrentCustomerSnapshot(supabase);
  let result = { status: "empty" };

  if (hasLookup) {
    try {
      result = await findPublicOrderTracking({ contact, orderNumber });
    } catch (error) {
      result = {
        message: error.message,
        status: "setup-required"
      };
    }
  }

  return (
    <main className={cx(globalStyles, "page-shell auth-page tracking-page")}>
      <SiteHeader user={snapshot.user} />

      <section className={cx(globalStyles, "tracking-layout")}>
        <form className={cx(globalStyles, "auth-card tracking-lookup-card")} method="GET">
          <p className={cx(globalStyles, "section-label")}>Rastreio TSZR15</p>
          <h1>Acompanhe seu pedido.</h1>
          <p className={cx(globalStyles, "helper-text")}>
            Use o numero gerado no checkout e o WhatsApp da compra.
          </p>

          <label>
            <span>Numero do pedido</span>
            <input defaultValue={orderNumber} name="pedido" placeholder="TSZ-..." required />
          </label>

          <label>
            <span>WhatsApp ou CPF/CNPJ</span>
            <input defaultValue={contact} name="contato" required />
          </label>

          <button className={cx(globalStyles, "button button-primary")} type="submit">
            Consultar rastreio
          </button>
        </form>

        <TrackingResult result={result} />
      </section>
    </main>
  );
}
```

10. Estilo, assets e APIs do navegador

A camada visual usa CSS global em app/globals.css, CSS Module pontual para componente de senha, imagens em public/brand e URLs de Storage Supabase para assets remotos.

Recurso	Arquivo	Uso
CSS global	app/globals.css	Layout da loja, botoes, cards, admin, rastreio, carrinho e responsividade.
CSS Module	src/components/form/password-input.module.css	Escopo local do input de senha.
public assets	public/brand/*.png	Logo e imagens de marca versionadas no repo.
Supabase Storage	heroBoardSrc e buckets product-images/review-photos	Imagens que podem vir do painel/admin ou do bucket publico.
Web APIs	localStorage, fetch, AbortController, window.open	Carrinho, chamadas internas, consulta de CEP e abertura do WhatsApp.
Intl	Intl.NumberFormat, Intl.DateTimeFormat	Moeda e datas em pt-BR.

src/customer/cep-lookup.js - ViaCEP e AbortController

```
import { digitsOnly, sanitizeCep, sanitizeState } from "./field-validation.js";

const viaCepBaseUrl = "https://viacep.com.br/ws";

function cleanText(value) {
  return String(value ?? "").trim();
}

export function getCepDigits(value) {
  return digitsOnly(value).slice(0, 8);
}

export function canLookupCep(value) {
  return getCepDigits(value).length === 8;
}

export function buildViaCepLookupUrl(value) {
  const cepDigits = getCepDigits(value);

  if (cepDigits.length !== 8) {
    return "";
  }

  return `${viaCepBaseUrl}/${cepDigits}/json/`;
}

export function normalizeViaCepAddress(data) {
  if (!data || data.erro) {
    return null;
  }

  const state = sanitizeState(data.uf);

  return {
    cep: sanitizeCep(data.cep),
    city: cleanText(data.localidade),
    district: cleanText(data.bairro),
    state,
    street: cleanText(data.logradouro)
  };
}

export function formatCepAddressLine(address) {
  if (!address) {
    return "";
  }

  const cityLine = [address.city, address.state].filter(Boolean).join("/");

  return [address.street, address.district, cityLine].filter(Boolean).join(" - ");
}

export async function fetchCepAddress(value, { signal } = {}) {
  const lookupUrl = buildViaCepLookupUrl(value);

  if (!lookupUrl) {
    return null;
  }

  const response = await fetch(lookupUrl, { signal });

  if (!response.ok) {
    throw new Error("Nao foi possivel consultar o CEP agora.");
  }

  return normalizeViaCepAddress(await response.json());
```

O ViaCEP nao exige chave. O projeto monta a URL apenas quando o CEP tem 8 digitos, normaliza a resposta e deixa o componente cliente abortar requests antigos quando o usuario altera o campo rapidamente.

11. Scripts, testes e qualidade

O projeto usa scripts npm simples para desenvolvimento, build, lint, formatacao, testes e sincronizacao de catalogo. Os testes usam node:test e assert/strict, sem dependencia extra de test runner.

package.json - scripts principais

```
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "build:cf": "opennextjs-cloudflare build",
    "deploy": "opennextjs-cloudflare build && opennextjs-cloudflare deploy",
    "preview": "opennextjs-cloudflare build && opennextjs-cloudflare preview",
    "format": "prettier --write \"app/api/**/*.js\" \"app/auth/actions.js\"
  \"app/page.js\" \"app/catalogo/page.js\" \"app/pedido/page.js\"
  \"app/produto/[slug]/page.js\" \"src/components/catalog/*.js\" \"src/lib/*.js\"
  \"tests/rate-limit.test.mjs\" \"*.{json,mjs}\"",
    "format:check": "prettier --check \"app/api/**/*.js\" \"app/auth/actions.js\"
  \"app/page.js\" \"app/catalogo/page.js\" \"app/pedido/page.js\"
  \"app/produto/[slug]/page.js\" \"src/components/catalog/*.js\" \"src/lib/*.js\"
  \"tests/rate-limit.test.mjs\" \"*.{json,mjs}\"",
    "lint": "eslint app src tests scripts next.config.mjs",
    "start": "next start",
    "clone:preview": "node scripts/clone-supabase-preview.mjs",
    "sync:catalog": "node scripts/sync-supabase-catalog.mjs",
    "sync:catalog:preview": "node scripts/sync-supabase-catalog.mjs --target preview",
    "optimize:brand-assets": "node scripts/optimize-brand-assets.mjs",
    "optimize:product-images": "node scripts/optimize-product-images.mjs --dry-run",
    "optimize:product-images:apply": "node scripts/optimize-product-images.mjs
  --apply",
```

scripts/validate-catalog.mjs - validacao local do catalogo

```
import { catalogProducts, rejectedCatalogProducts, validateCatalog } from
  "../src/catalog/index.js";

const issues = validateCatalog();

if (issues.length > 0) {
  console.error("Catalog validation failed:");
  for (const issue of issues) {
    console.error(`- ${issue}`);
  }
  process.exit(1);
}

console.log(`Catalog validation passed for ${catalogProducts.length} published
  products.`);
console.log(`Rejected products: ${rejectedCatalogProducts.length}.`);
```

scripts/sync-supabase-catalog.mjs - seed/upsert do Supabase

```
171 | async function main() {
172 |   loadLocalEnv();
173 |
174 |   if (targetArg) {
175 |     process.env.SUPABASE_RUNTIME_TARGET = targetArg;
176 |   }
177 |
178 |   const catalogIssues = validateCatalog();
179 |
180 |   if (catalogIssues.length > 0) {
181 |     throw new Error(`Catalogo local invalido:\n${catalogIssues.join("\n")}`);
182 |   }
183 |
184 |   const categoryRows = buildCatalogCategoryRows();
185 |   const productRows = buildCatalogProductRows();
186 |   const relationRows = buildCatalogProductCategoryRows();
187 |
188 |   if (dryRun) {
189 |     console.log(
190 |       `Dry run: ${categoryRows.length} categorias, ${productRows.length} produtos e
    ${relationRows.length} vinculos prontos para sincronizar.`
191 |     );
192 |     return;
193 |   }
194 |
195 |   if (sqlOutputPath) {
196 |     const outputPath = resolve(process.cwd(), sqlOutputPath);
197 |
198 |     mkdirSync(resolve(outputPath, ".."), { recursive: true });
199 |     writeFileSync(outputPath, buildSeedSql({ categoryRows, productRows, relationRows
    }), "utf8");
200 |     console.log(`SQL de seed gerado em ${sqlOutputPath}.`);
201 |     return;
202 |   }
203 |
204 |   const url = getSupabaseUrl();
205 |   const serviceRoleKey = getSupabaseServiceRoleKey();
206 |
207 |   if (!url || !serviceRoleKey) {
208 |     throw new Error(
209 |       "Configure as envs Supabase do destino. Para preview use
    NEXT_PUBLIC_SUPABASE_PREVIEW_URL e SUPABASE_PREVIEW_SERVICE_ROLE_KEY, ou rode com as envs
    padrao."
210 |     );
211 |   }
212 |
213 |   if (getJwtRole(serviceRoleKey) === "anon") {
214 |     throw new Error(
215 |       "SUPABASE_SERVICE_ROLE_KEY esta com uma chave anon. Use a service_role real ou
    gere SQL com: npm run sync:catalog -- --write-sql supabase/.temp/catalog-seed.sql"
216 |     );
217 |   }
218 |
219 |   const client = createClient(url, serviceRoleKey, {
220 |     auth: {
221 |       autoRefreshToken: false,
222 |       persistSession: false
```

tests/rate-limit.test.mjs - exemplo de teste node:test

```
 1 | import assert from "node:assert/strict";
 2 | import { afterEach, test } from "node:test";
 3 |
 4 | import { redactSensitive } from "../src/lib/logger.js";
 5 | import {
 6 |   clearLocalRateLimitStore,
 7 |   consumeRateLimit,
 8 |   hashRateLimitIdentifier,
 9 |   rateLimitProfiles
10 | } from "../src/lib/rate-limit.js";
11 | import { createRateLimitResponse } from "../src/lib/rate-limit-response.js";
12 |
13 | afterEach(() => {
14 |   clearLocalRateLimitStore();
15 |   delete process.env.VERCEL_ENV;
16 | });
17 |
18 | test("rate limit permite requisicoes dentro da janela e bloqueia acima do limite",
    async () => {
19 |   const profile = {
20 |     blockSeconds: 30,
21 |     limit: 2,
22 |     scope: "test-checkout",
23 |     windowSeconds: 60
24 |   };
25 |
26 |   const first = await consumeRateLimit({ ...profile, identifier: "127.0.0.1", supabase:
    null });
27 |   const second = await consumeRateLimit({ ...profile, identifier: "127.0.0.1",
    supabase: null });
28 |   const third = await consumeRateLimit({ ...profile, identifier: "127.0.0.1", supabase:
    null });
29 |
30 |   assert.equal(first.allowed, true);
31 |   assert.equal(second.allowed, true);
32 |   assert.equal(third.allowed, false);
33 |   assert.equal(third.retryAfterSeconds, 30);
34 | });
35 |
36 | test("rate limit separa chaves por rota, IP e cupom", async () => {
37 |   const profile = {
38 |     blockSeconds: 30,
39 |     limit: 1,
40 |     scope: "coupon-a",
41 |     windowSeconds: 60
42 |   };
43 |
44 |   await consumeRateLimit({ ...profile, identifier: "ip:cupom-a", supabase: null });
```

eslint.config.mjs - lint JavaScript/JSX

```
import js from "@eslint/js";
import prettier from "eslint-config-prettier/flat";
import globals from "globals";

export default [
  {
    ignores: [".next/**", "node_modules/**", "coverage/**", "supabase/.temp/**"]
  },
  js.configs.recommended,
  {
    languageOptions: {
      ecmaVersion: "latest",
      globals: {
        ...globals.browser,
        ...globals.node
      },
      parserOptions: {
        ecmaFeatures: {
          jsx: true
        }
      },
      sourceType: "module"
    },
    rules: {
      "no-control-regex": "off",
      "no-useless-assignment": "warn",
      "no-unused-vars": [
        "warn",
        {
          argsIgnorePattern: "^_",
          caughtErrorsIgnorePattern: "^_",
          varsIgnorePattern: "^_"
        }
      ]
    }
  },
  prettier
];
```

.github/workflows/secret-scan.yml - Gitleaks no CI

```
name: Secret scan

on:
  pull_request:
  push:
    branches:
      - main
      - MODEL_WHATSBUIS
  workflow_dispatch:

permissions:
  contents: read
  pull-requests: read

jobs:
  gitleaks:
    name: Gitleaks
    runs-on: ubuntu-latest
    steps:
      - name: Checkout full history
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Install Gitleaks
        shell: bash
        run: |
          set -euo pipefail
          version="8.30.1"
          asset="gitleaks_${version}_linux_x64.tar.gz"
          base_url="https://github.com/gitleaks/gitleaks/releases/download/v${version}"
          curl -sSfL "${base_url}/${asset}" -o "/tmp/${asset}"
          curl -sSfL "${base_url}/gitleaks_${version}_checksums.txt" -o
    /tmp/gitleaks_checksums.txt
```

12. Configuracao e deploy

A configuracao do projeto e feita por variaveis de ambiente. O arquivo .env.example mostra nomes esperados sem segredos reais. O deploy de producao roda em Cloudflare Workers via OpenNext (@opennextjs/cloudflare): npm run deploy faz o build (opennextjs-cloudflare build) e publica o worker; wrangler.jsonc define o worker, os assets estaticos e os bindings de KV (cache incremental), D1 (tag cache) e Durable Object (fila de revalidacao). O codigo ainda detecta Vercel Preview: prefere credenciais de Supabase Preview quando VERCEL_ENV=preview ou SUPABASE_RUNTIME_TARGET=preview.

.env.example - variaveis usadas pelo app

```
NEXT_PUBLIC_WHATSAPP_BUSINESS_NUMBER=5511999999999
WHATSAPP_BUSINESS_NUMBER=5511999999999
NEXT_PUBLIC_STORE_NAME=TSZR15
NEXT_PUBLIC_SITE_URL=https://www.tszr15-store.com.br
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
# Fallbacks used when a hosted integration provides server-side names only.
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_SECRET_KEY=
# Optional isolated Supabase project for Vercel Preview / test branches.
# Used automatically when VERCEL_ENV=preview, or locally with
SUPABASE_RUNTIME_TARGET=preview.
SUPABASE_RUNTIME_TARGET=
NEXT_PUBLIC_SUPABASE_PREVIEW_URL=
NEXT_PUBLIC_SUPABASE_PREVIEW_PUBLISHABLE_KEY=
SUPABASE_PREVIEW_URL=
SUPABASE_PREVIEW_PUBLISHABLE_KEY=
SUPABASE_PREVIEW_ANON_KEY=
SUPABASE_PREVIEW_SERVICE_ROLE_KEY=
SUPABASE_PREVIEW_SECRET_KEY=
TSZR15_ADMIN_TOKEN=
```

Grupo	Variaveis	Papel
WhatsApp	NEXT_PUBLIC_WHATSAPP_BUSINESS_NUMBER, WHATSAPP_BUSINESS_NUMBER	Numero usado na URL wa.me.
Supabase publico	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Auth e leituras publicas/autenticadas.
Supabase servidor	SUPABASE_SERVICE_ROLE_KEY, SUPABASE_SECRET_KEY	Pedidos de convidado, admin, storage, cupons e rastreio.
Preview	NEXT_PUBLIC_SUPABASE_PREVIEW_URL, SUPABASE_PREVIEW_SERVICE_ROLE_KEY	Projeto isolado para preview/teste.
Admin	TSZR15_ADMIN_TOKEN	Senha/token do painel interno.
Site	NEXT_PUBLIC_SITE_URL, SITE_URL, VERCEL_URL	Origem segura para reset de senha e callbacks.

13. Tecnologias planejadas ou documentadas para evolucao

docs/APIS-NECESSARIAS.md lista integracoes para evoluir o MVP. Elas nao devem ser confundidas com features ja implementadas no codigo atual.

Tecnologia	Possivel uso	Status
Mercado Pago	Pagamento Pix/link/cartao e webhooks de confirmacao.	Planejado
WhatsApp Cloud API	Mensagens oficiais, webhooks e templates.	Planejado

Tecnologia	Possivel uso	Status
Shopee Open Platform	Automacao futura somente se o fluxo for permitido oficialmente.	Planejado
AliExpress/DSers	Fornecedor ou integrador homologado para compra assistida.	Planejado
Correios/Melhor Envio	Frete, prazo, etiquetas ou rastreio quando houver estoque proprio.	Opcional
Resend	Email transacional para comprovantes e atualizacoes.	Planejado
Nota fiscal/ERP	Explicitamente fora do escopo planejado atual.	Fora do escopo

docs/APIS-NECESSARIAS.md - ordem recomendada

```
286 | ## Ordem recomendada de implementacao
287 |
288 | 1. Banco de dados e pedido salvo.
289 | 2. Consentimento de compra assistida.
290 | 3. Painel admin para registrar compra em Shopee/AliExpress manualmente.
291 | 4. Mercado Pago para pagamento confirmado.
292 | 5. ViaCEP para reduzir erro de endereco.
293 | 6. Rastreio manual e notificacao por WhatsApp.
294 | 7. Automacao de Shopee/AliExpress somente depois de validar acesso oficial ou
    integrador homologado.
```

14. Roteiro de estudo do projeto

- Comece em README.md para entender produto, rotas e variaveis.
- Leia package.json para ver stack e scripts executaveis.
- Abra app/layout.js, app/page.js e app/api/checkout/whatsapp/route.js para entender o App Router.
- Siga src/catalog e src/components/catalog para entender vitrine, busca, produto e carrinho.
- Siga src/checkout para entender totais, cupom, validacao e mensagem WhatsApp.
- Leia src/lib/supabase e supabase/migrations para entender dados, Auth, RLS, RPC e Storage.
- Leia src/admin e app/admin/actions.js para entender o backoffice e revalidacao.
- Rode npm run validate e npm test quando as dependencias estiverem instaladas corretamente.

Checklist mental

Pergunte sempre: este dado vem do cliente ou do servidor? Ele pode ir para o browser? Ele depende de service_role? A resposta define se o codigo deve ficar em componente cliente, Server Action, API route, proxy ou RPC SQL.